Layer10 Take-Home Project

Grounded Long-Term Memory via Structured Extraction, Deduplication, and a Memory Graph

Krisha Biradar : 24B3957

March 8, 2026

1 Corpus

1.1 Dataset Used

I used the public GitHub issue tracker for the `psf/requests` library. The corpus was fetched live via the GitHub REST API, collecting 40 real issues along with all their associated comments.

This corpus is a good is a good and diverse dataset containing bug reports, feature proposals, decisions, status transitions (open → closed), assignee changes, cross-references between issues, and multi-person technical discussions.

1.2 How to Reproduce the Download

No manual download is required sincethe notebook fetches data automatically from the GitHub API while the notebook is running. A free GitHub personal access token is required to avoid rate limits (60 requests/hour without a token, 5000/hour with one).

```
Owner  : psf
Repo   : requests
Issues : 40 (state=all, i.e. open + closed)
API    : https://api.github.com/repos/psf/requests/issues
```

2 Ontology and Schema

2.1 Entity Types

Totally five entity types have been defined to capture the key objects in the corpus.

Entity Type	Description
Issue	A GitHub issue identified by number and title
Person	A GitHub user - could be author, commenter, or assignee
Label	A tag applied to an issue (e.g. *bug*, *enhancement*)
State	Status of an issue: *open* or *closed*
Concept	A topic, problem, or solution extracted by the LLM from issue text

2.2 Relation Types

Totally nine relation types split across the 2 extraction phases.

Relation	Meaning
CREATED_BY	Issue was opened by this person
ASSIGNED_TO	Issue is assigned to this person
HAS_STATE	Current or historical state of the issue
HAS_LABEL	Label applied to the issue
COMMENTED_ON	Person left a comment on the issue
MENTIONS_USER	Text mentions another GitHub user
REFERENCES_ISSUE	Text references another issue by number
PROPOSES_SOLUTION	Text proposes a fix or workaround
REPORTS_PROBLEM	Text describes a bug or problem

The first five relations come from rule-based extraction pass and last four are extracted by LLM.

3 Structured Extraction

Extraction runs in two sequential phases. Totally 336 claims are extracted

139 rule-based claims = the stuff GitHub told us directly.

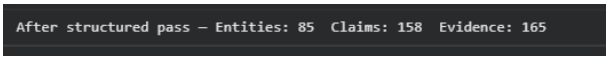


Figure 1: proof from notebook

197 LLM claims = facts that only exist in the free text that is not strucutred.

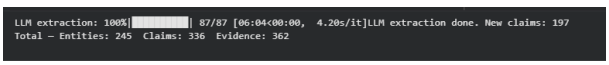


Figure 2: proof

3.1 Phase 1: Rule-Based Extraction

Since the GitHub API response directly gives a structured response that already has specific fields filled in(like state, assignees, and labels) so these are extracted directly with

no model calls. Confidence is always 1.0 because the data is explicit and unambiguous.

State transitions are handled carefully. When an issue is closed 2 claims are created:

- `HAS_STATE = open`, `valid_from = created_at`, `valid_to = closed_at`
- `HAS_STATE = closed`, `valid_from = closed_at`, `valid_to = null`

This preserves both the current state and the full history of transitions in the memory graph.

3.2 Phase 2: LLM Extraction (Groq)

For this part LLM is used and I used `llama-3.1-8b-instant` via the free Groq API to extract "meaning-based or text-derived" claims from issue bodies and comments. In the prompt the model is asked to return a JSON array of claims each containing a relation type, object, a verbatim supporting excerpt from the text, and a confidence.

3.2.1 Validation and Repair

For validation first the raw LLM output goes through the following pipeline before being accepted:

1. Try to parse the response as JSON
2. If parsing fails search the raw text for a JSON array (repair step)
3. Strip markdown code fences if present
4. Validate that each claim has all required fields
5. Reject claims with confidence < 0.5 (qquality check)
6. Reject claims with unrecognised relation types
7. Retry up to 2 times on failure

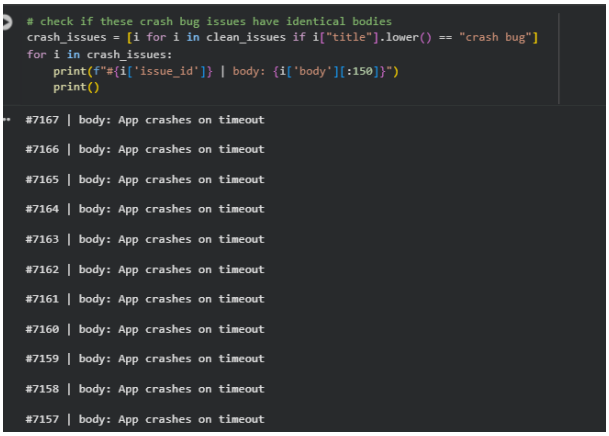
Only non duplicate artifacts with more than 15 tokens are sent to the LLM this reduces unnecessary API calls.

3.2.2 Confidence

Rule-based claims carry confidence = 1.0 because the data comes directly from the GitHub API with no ambiguity. LLM-extracted claims carry whatever confidence the model assigns based on how much it had to read into the text. Minimum is 0.5, we reject the others. In our corpus the LLM mostly assigned high confidence because most issue text is direct and explicit. However it still did a good job of extracting.

3.2.3 Extraction Versioning

Every claim stores an `extraction_version` string in format `schema_version|method|model`, for example `v1.0|structured` or `v1.0|llm|llama-3.1-8b-instant`. If the ontology changes or a new model is used old claims can be identified and updated without losing the original extraction history.

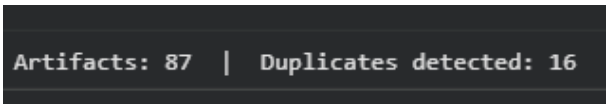


```
# check if these crash bug issues have identical bodies
crash_issues = [i for i in clean_issues if i["title"].lower() == "crash bug"]
for i in crash_issues:
    print(f"#{i['issue_id']} | body: {i['body'][:150]}")
    print()

#7167 | body: App crashes on timeout

#7166 | body: App crashes on timeout

#7165 | body: App crashes on timeout

#7164 | body: App crashes on timeout

#7163 | body: App crashes on timeout

#7162 | body: App crashes on timeout

#7161 | body: App crashes on timeout

#7160 | body: App crashes on timeout

#7159 | body: App crashes on timeout

#7158 | body: App crashes on timeout

#7157 | body: App crashes on timeout
```

Figure 4: "App crashes on timeout"

4 Deduplication and Canonicalization

4.1 Artifact-Level Deduplication



```
Artifacts: 87  |  Duplicates detected: 16
```

Figure 3: Number of artifacts and duplicates

Before any extraction every artifact (issue body or comment) is checked for duplicates at two levels:

- **Exact match:** SHA-256 hash of the text content. If the hash already exists in the store the artifact is flagged as a duplicate.
- **Near-duplicate:** Jaccard similarity on word tokens. If overlap exceeds 0.85 for texts longer than 10 words the artifact is flagged as a near-duplicate.

Duplicates are not deleted. They are stored with `is_duplicate = True` and a `duplicate_of` pointer to the canonical artifact. No claims are extracted from duplicate artifacts. This makes the deduplication fully reversible since no information is destroyed.

4.1.1 Real Example from the Corpus

```
# check if they got flagged as duplicates
for i in crash_issues:
    art_id = f"issue_{i['issue_id']}_body"
    art = artifact_store[art_id]
    print(f"#{i['issue_id']} | is_duplicate: {art['is_duplicate']} | duplicate_of: {art['duplicate_of']}")

#7167 | is_duplicate: False | duplicate_of: None
#7166 | is_duplicate: True | duplicate_of: issue_7167_body
#7165 | is_duplicate: True | duplicate_of: issue_7167_body
#7164 | is_duplicate: True | duplicate_of: issue_7167_body
#7163 | is_duplicate: True | duplicate_of: issue_7167_body
#7162 | is_duplicate: True | duplicate_of: issue_7167_body
#7161 | is_duplicate: True | duplicate_of: issue_7167_body
#7160 | is_duplicate: True | duplicate_of: issue_7167_body
#7159 | is_duplicate: True | duplicate_of: issue_7167_body
#7158 | is_duplicate: True | duplicate_of: issue_7167_body
#7157 | is_duplicate: True | duplicate_of: issue_7167_body
```

Figure 5: Duplicates and proof they got flagged

Issues #7157 to #7167 all have the title "Crash bug" and identical body text

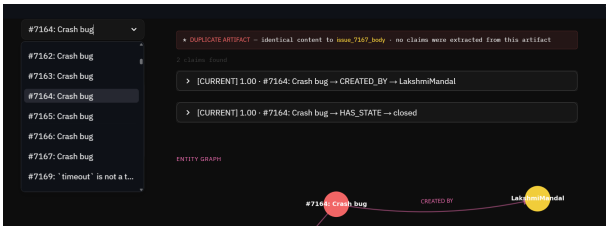


Figure 6: Screenshot from UI showing its working

"App crashes on timeout." The artifact deduplication correctly identifies #7167 as the canonical artifact and flags all others as duplicates pointing to it. The issues as still kept as separate entities in the graph since they were genuinely filed separately by different users, but no redundant claims are being extracted from them. Above is an example of content - duplication, where the body was the same but the entity names were different (all had different numbers).

4.2 Entity Canonicalization

Entities are deduplicated at creation time using a case-folded key `(entity_type, name.lower())`. This basically handles those cases when the enitity is the same but has 2 or more different names due to capital letters causing name variations. These are resolved automatically.

For near duplicate names alias candidates are detected by counting shared characters between names. Pairs above a similarity threshold are flagged and merged.

Merges are reversible. The dropped entity is not deleted but it receives a `merged_into` pointer and a `merge_reason` tag, and all its claims are repointed to the canonical entity. Reversing a merge just requires removing the pointer and repointing claims.

This method is applied to both `Person` and `Concept` entity types.

Here is an example of "davidism" both a concept and person have the same enitity name "davidism" but their body is different.

Figure 7: Enter Caption

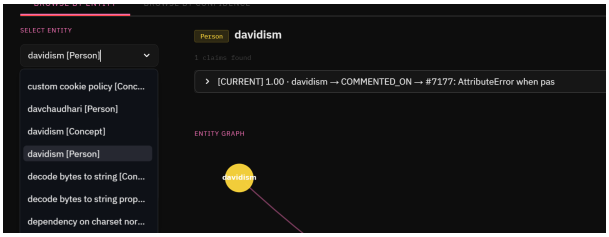


Figure 8: Person "davidism" in the UI

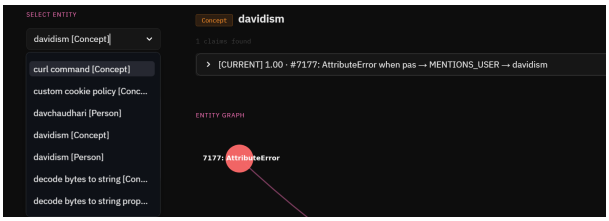


Figure 9: Concept "davidism" in the UI

4.3 Concept Deduplication

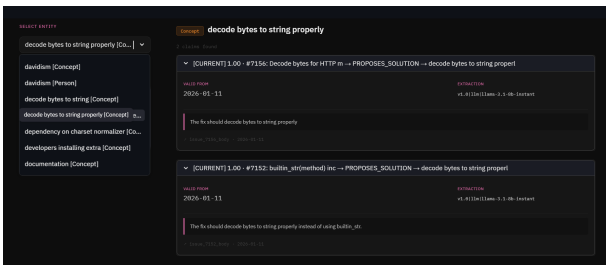
The same approach is applied to Concept entities but with a slightly lower threshold to catch plurals and minor variations (like for example "things" vs "thing"). However there is a limitation, purely lexical comparison misses meaning based near-duplicates for example something like, "thing" vs "same thing" they are same concept but are different as strings.

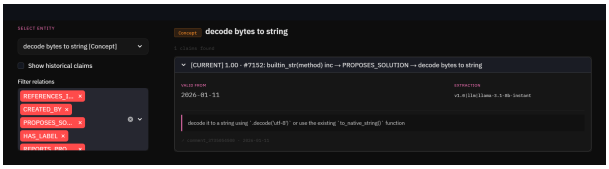
There is an example of the limitations in the corpus:

"decode bytes to string" and "decode bytes to string properly" so for them the entity name is different enough that lexical similarity fails to flag them but in reality according to the body are the same concept and should have been merged into one. An embedding model would place both in nearly the same vector space and correctly merge them and I would acknowledge this as future work.

4.4 Claim Deduplication

Claims are deduplicated using a composite key: `(subject_id, relation, object_id)`. If the same fact is extracted from multiple artifacts only one claim is stored and all evidence is accumulated to it. ALso each additional piece of evidence slightly increases the confidence of that claim (but obviously capped at 1), it just shows that multiple independent sources strengthen a fact.





5 Memory Graph Design

5.1 Core Objects

The memory graph stores the following four types of objects:

- **Entity:** `entity_id`, `entity_type`, `canonical_name`, `aliases`, optional `merged_into`
- **Claim:** `subject`, `relation`, `object`, `event_time`, `valid_from`, `valid_to`, `confidence`, `extraction_version`, `evidence_ids`
- **Evidence:** `artifact_id`, `excerpt`, `offset_start`, `timestamp`, `url`
- **Artifact:** raw source text with `is_duplicate` and `duplicate_of` fields

5.2 Time Model

Every claim has two time fields:

- `event_time`: when the source message was created
- `valid_from` / `valid_to`: the time interval during which the claim was true. `valid_to = null` means the claim is still current.

When a fact changes (example, an issue moves from open to closed) the old claim gets `valid_to` set to the transition time and a new claim is created for the new state. This is to preserve full history without overwriting anything.

5.3 Updates and Idempotency

Ingestion works the same way every time. If you run the pipeline twice on the same data, you'll get the same graph. Thats because the system checks artifact hashes before storing anything, looks up entities by their type and name (`(type, name)`) before creating them, and checks claims by their composite keys.

6 Retrieval and Grounding

6.1 How Retrieval Works

It works in three steps:

1. **Entity search:** find entities whose canonical name or aliases contain the query as a substring. Exact name matches then score 2. Alias matches then score 1.
2. **Claim expansion:** collect all claims connected to matched entities. By default only current claims (`valid_to = null`) are returned. Setting `history=True` includes historical claims as well.

3. **Ranking:** sort by currency (current before historical), then confidence descending, then recency descending. Return the top 10 results.

Every returned item is grounded so it carries a verbatim excerpt from the original artifact and a direct link to the GitHub source page. Nothing is returned without evidence.

6.2 Example Context Packs

Context packs were generated for six queries and saved to `context_examples.json`:

Query	Entity Type
`jmdotdev`	Person
`shelvacu`	Person
`timeout errors`	Concept
`requests library`	Concept
`bug`	Label
`closed`	State

7 Visualization

7.1 Overview Graph

A static overview graph is produced in the notebook showing the top 50 nodes by degree, it gives a high-level picture of the most connected entities.

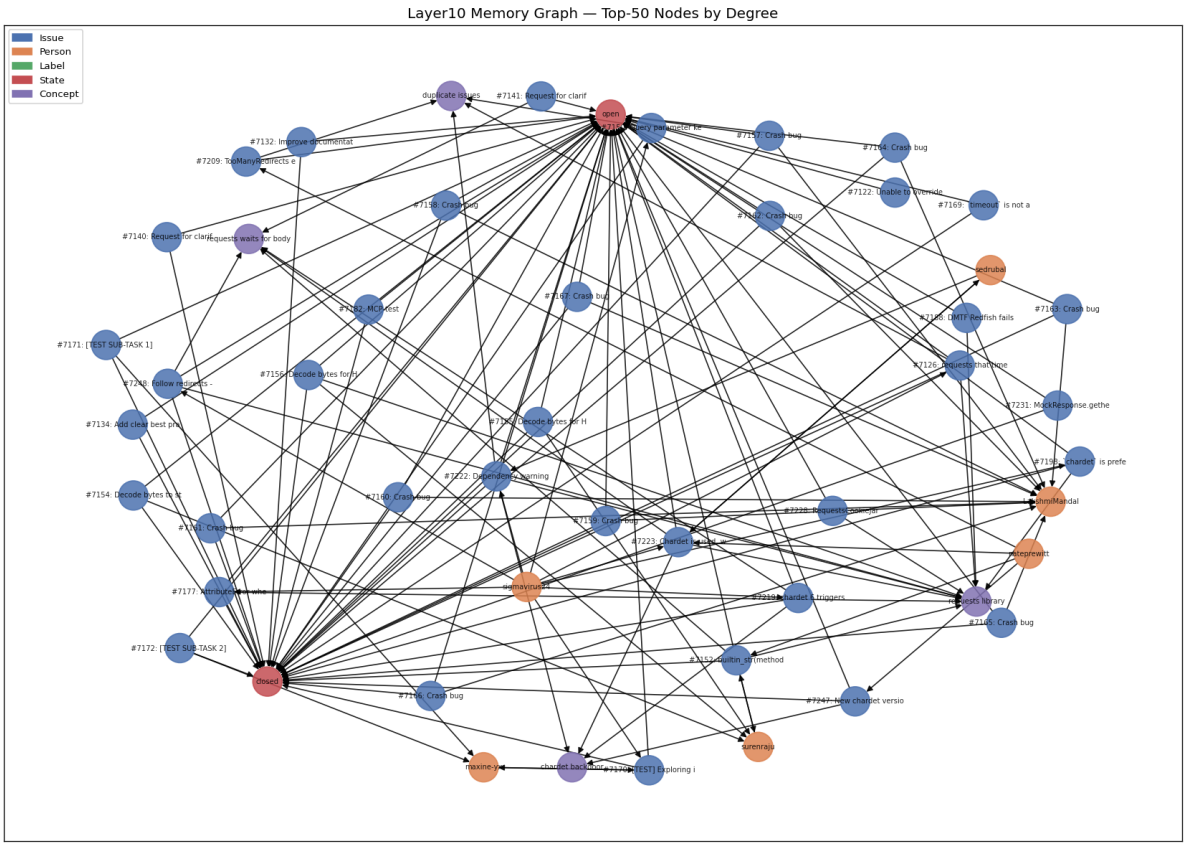


Figure 10: Memory graph overview

7.2 Streamlit Interactive UI

The Streamlit app provides two views.

7.2.1 Browse by Entity

Pick any entity from the dropdown, and youll see its type badge, any aliases it has, whether its a duplicate or the main (canonical) artifact, every related claim (tagged as either CURRENT or HISTORICAL).

There is a evidence panel for each claim with supporting text and a clickable source link.

Below there will also be an entity graph for that particular entity showing connections with appropriate labelling

7.2.2 Browse by Confidence

Browse by Confidence lets you look at all claims in the graph, sorted from lowest to highest confidence. You can use a slider to set the maximum confidence you want to see. I just felt this view would probably really helps when auditing low-confidence extractions and want to see where the LLM got shaky or uncertain.

8 Layer10 Adaptation

8.1 Ontology Changes

For Layer10's real environment (email, Slack, Jira, Linear), the schema would need several additions:

- **Channel** : since on Slack conversations happen in channels which are basically like group chats. On email, this would be a distribution list. On Jira a project. The channel is just the container in which conversations are happening.
- **Thread/Chain** : inside a channel people would reply to specific messages in response to it so it will automatically create a thread (chain of replies). So it would be better to keep the thread or chain as a separate enitity since it holds a full conversation.
- **Decision*** : sometimes a conversation ends with someone saying "ok we are going with option A." That decision should be stored as its own entity because it affects future behaviour, and it would be more useful to just filter from decision enitity in most cases. It should probably be marked as reversible entity since someone could just say "actually we changed our minds.". But I think this would be quite hard to implement due to the reversibility
- **Ticket** — in Jira or Linear every task or bug has a ticket which has a number like `JIRA-1234`. It has a status, a priority, and a sprint. So shoulde be a separate entity.
- **Document** — if someone shares a Google Doc or Notion page, that document should be an entity so that the system knows it exits, in which conversations it was linked to.

New relation types would include something like `LINKS_TO`, `BLOCKS`, `DUPLICATES`, `DECIDED_IN`, and `REVERSED_BY`.

8.2 Unstructured and Structured Fusion

When a Slack message references a Jira ticket, a `DISCUSSED_IN` edge should connect Ticket entity to Thread entity. cross-references. Ownership changes in Jira produce `ASSIGNED_TO` claims with `valid_to` set on the previous claim. This part is more or else the same as the github issues Corpus used in my project.

8.3 Memory Worthiness

Not everything should be stored in memory forever since there will be multiple messages that are just filler, for example someone replying with "ok, will do". This would just waste storage and clutter so it would perhaps be better to delete these when they get extracted as a claim by the LLM.

We could do some thing like "Relation-based filtering" - after extraction if the only claim extracted is something like person COMMENTED_ON issue with no meaning full content, don't store it. Only store claims that have actual informational relations like REPORTS_PROBLEM, PROPOSES_SOLUTION, DECIDED_IN.

8.4 Deletion cases

Sometimes in some conversation, if a person deleted a controversial message that they probably sent by mistake and would not want it to be recorded, we would need it to be deleted from the memory system too. So we would need to delete that claim without affecting anything else.

8.5 Permissions

Permissions, at least in theory, are pretty straightforward. Every artifact would have its own access control list (acl) with user or group IDs. When someone tries to retrieve evidence, the system only shows items where that user is on the artifact's acl. If a claim is backed only by evidence the user isn't allowed to see, the claim just stays hidden.

Like for example there is some message in #HR-private that says "we're planning to fire John soon" That message will get ingested and a claim gets, so when someone queries the memory system asking "what's happening with John" obviously not everyone should be able to see that message. Only people who had access to #HR-private should be able to see it.

8.6 Observability

For observability, the memory export comes with a metadata block—counts of artifacts, entities, claims, and evidence, along with the schema version and model info. In a real production setup, we'd also keep track of extraction failures, repair rates, confidence scores, and how long each type of query takes. This helps spot when quality starts to slip.

9 Reproducibility

9.1 Dependencies

```
pip install requests networkx groq tqdm pyngrok streamlit
Python 3.10+
```

9.2 Step-by-Step Instructions

1. Open `layer10_final.ipynb` in Google Colab
2. In the Config cell (First cell in the notebook "Setup") and fill in:
   - `GROQ_API_KEY` : need to make free account at https://console.groq.com
   - `GITHUB_TOKEN` : free personal access token at https://github.com/settings/tokens
3. In the Streamlit launch cell (Almost at the bottom, just before "Proofs for WriteUp" block) and add your ngrok token (free at https://dashboard.ngrok.com)
4. Go to **Runtime** → **Run all**

5. Wait approximately 6-7 minutes (LLM extraction takes time) for the whole code to finish running.
6. Click on the URL generated in the same Streamlit launch cell. You will be able to interact with the UI.